Command sequence in `main_interfaz.f90`

This document describes the sequence of operations performed by `main_interfaz` (source file `main_interfaz.f90`). When the executable is run, the user is guided through the steps below via a series of prompts in the terminal. Inputs are read from standard input through the helper subroutine `read_clean_line`, which skips blank lines and full-line comments (lines whose first non-blank character is "!" or "#"), so the input file can be annotated and reused via input redirection:

```
> .\interfaz_remix.exe < datos_interfaz_denit_2reacts.txt
```
(run from cmd.exe)

All error/EOF conditions are handled gracefully and any non-positive time step is rejected. The program executes exactly one reactive-mixing iteration per invocation (re-run the program, with an updated `u_tilde` file, for each new iteration).

Pre-process

1. Query IEEE floating-point support. If available, switch to abrupt underflow (`ieee_set_underflow_mode` with `gradual = .false.`) and clear the underflow, inexact, overflow, divide-by-zero and invalid exception flags so that benign exceptions do not abort the run.
2. Print introductory message: *"This is the interface to solve the reactive part of a 1D reactive mixing iteration using explicit Euler."*
3. Create chemistry class object `my_chem`.

Input reading

4. Read the database path (`path_DB`), including a trailing backslash or forward slash depending on the OS. Prompt: *"Database path:"*.
5. Read the problem path (`path_pb`), with the corresponding trailing slash. Prompt: *"Problem path:"*.
6. Read the common root of the input and output file names (`root_files`). Prompt: *"Root of the input and output files:"*.
7. Read the number of targets in the mesh (`num_tar`) in response to the prompt *"How many targets does the mesh have?"*.
8. Read chemistry: `my_chem` calls `read_chemistry_interface`, passing the trimmed root, problem path, database path and `num_tar`. The latter is used to validate the target waters file against the mesh size.
9. Read the flag `flag_wat_types` indicating whether the file with the aqueous component concentrations of the initial and external water types has already been generated in a previous run (`flag_wat_types = 1`) or must be generated now (`flag_wat_types = 0`). Any value other than 0 or 1 aborts execution with the error *"Invalid option. It must be 1 (file already generated) or 0 (generate it now)."*
10. Read the name of the water-types concentration file (`file_u_wat_types`). When `flag_wat_types = 0` it is the output filename where `my_chem` will write the aqueous component concentrations of the initial and external water types; when `flag_wat_types = 1` it is only used to display which existing file is being reused.
11. Get the number of aqueous components in the first target water: `my_chem` calls `get_num_aq_comps_tar_wat`. The value is stored in `num_aq_comps` and is used later to choose the interface and to read the `u_tilde` input file.

12. If `flag_wat_types = 0`, write the initial and external water-type concentrations: `my_chem` calls `write_conc_comp_tar_wat` and the program prints *"File <file_u_wat_types> generated successfully."* If `flag_wat_types = 1`, the program instead prints *"Using the existing file <file_u_wat_types>."*
13. Read the name of the file where `my_chem` will write the concentrations after the reactive mixing iteration (`file_u_new`).
14. Read the name of the file containing the aqueous component concentrations after solving an iteration of conservative transport (`file_u_tilde`). The file must live in the problem path.
15. Read the input-matrix layout flag (`flag_transpose`) indicating whether the `file_u_tilde` file has rows as targets and columns as components (`flag_transpose = 1`) or the canonical layout with rows as components and columns as targets (`flag_transpose = 0`). Any value other than 0 or 1 aborts execution with the error *"Invalid option. It must be 1 (transposed component matrix) or 0 (not transposed)."*
16. Read the time step (`Delta_t`). Execution is aborted with the error *"The time step must be greater than zero"* if the value is not strictly positive.

Process

17. Trim the `u_tilde` input filename and select the target of the procedure pointer `p_interfaz` according to the chemistry of the first target water and the orientation of the `u_tilde` file:
    - `interfaz_esp_arch` — if the first target water has no equilibrium reactions (kinetic-only system).
    - `interfaz_comps_arch_eq` / `interfaz_comps_arch_eq_T` — if there are equilibrium reactions but no kinetics (the `_T` variant is used when `flag_transpose = 1`).
    - `interfaz_comps_arch_eq_kin` / `interfaz_comps_arch_eq_kin_T` — if there are both equilibrium and kinetic reactions (the `_T` variant is used when `flag_transpose = 1`).
18. Print the message *"Proceeding with one reactive mixing iteration."*
19. Compute one reactive mixing iteration: call `p_interfaz` with `my_chem`, `path_pb`, `num_aq_comps`, `file_u_tilde`, `Delta_t` and `file_u_new`. The program performs a single iteration; to advance further, re-run the executable after updating `file_u_tilde` with the result of the next conservative-transport step.

Robust I/O and input comments

20. Every interactive read goes through the internal subroutine `read_clean_line(line, ios)`, which reads the next line from standard input into a 512-character buffer, skipping blank lines and lines whose first non-blank character is "!" or "#". The returned buffer is then parsed by an internal list-directed read of the form `read(buf, *, iostat=ios) <variable>`, which preserves the original type-safe conversion to integers, reals (e.g. `1d-2`) and character strings.
21. Only full-line comments are stripped: "!" or "#" appearing in the middle of a line are kept verbatim, so file paths containing those characters are preserved. Comments must therefore be placed on their own line.
22. Every interactive read is guarded by an `iostat` check on both the line-read and the internal read. On read failure or end-of-file, the program prints a message suggesting to run in an

interactive terminal (or redirect input from `fort.5`) and calls the internal `safe_stop(1)` subroutine, which clears pending IEEE flags before stopping with exit code 1.

Post-process

23. If IEEE support is available, clear the floating-point exception flags (via the internal `clear_ieee_flags` subroutine) before exiting.
24. Print termination message *"The program has finished."* and end the program.